

Programmation fonctionnelle 2

Preuves et programmes

CM12

Pierre Rousselin

Université Sorbonne Paris Nord

mai 2026

Fonctions d'ordre supérieur : rappels

Rappels sur la composition

`fold`

map sur les listes

- En OCaml

```
let rec map f l =  
  match l with  
  | [] -> []  
  | h :: t -> f h :: map f t
```

- Type?

map sur les listes

- En OCaml

```
let rec map f l =  
  match l with  
  | [] -> []  
  | h :: t -> f h :: map f t
```

- Type?

```
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

- En Rocq

map sur les listes

- En OCaml

```
let rec map f l =  
  match l with  
  | [] -> []  
  | h :: t -> f h :: map f t
```

- Type?

```
val map : ('a -> 'b) -> 'a list -> 'b list = <fun>
```

- En Rocq

```
Fixpoint map {A B : Type} (f : A -> B) (l : list A) :=  
  match l with  
  | [] => []  
  | h :: t => (f h) :: (map f t)  
end.
```

- Type :

```
map : (?A -> ?B) -> list ?A -> list ?B  
where  
?A : [ |- Type]  
?B : [ |- Type]
```

filter sur les listes

- En OCaml

```
let rec filter p l =  
  match l with  
  | [] -> []  
  | h :: t -> if p h then h :: filter p t else filter p t
```

- Type?

filter sur les listes

- ▶ En OCaml

```
let rec filter p l =  
  match l with  
  | [] -> []  
  | h :: t -> if p h then h :: filter p t else filter p t
```

- ▶ Type?

```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

- ▶ En Rocq

filter sur les listes

► En OCaml

```
let rec filter p l =  
  match l with  
  | [] -> []  
  | h :: t -> if p h then h :: filter p t else filter p t
```

► Type?

```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

► En Rocq

```
Fixpoint filter {A : Type} (p : A -> bool) (l : list A) :=  
  match l with  
  | [] => []  
  | h :: t => if p h then h :: filter p t else filter p t  
  end.
```

► Type?

```
filter : (?A -> bool) -> list ?A -> list ?A  
where  
?A : [ |- Type]
```


Exemples

► **Compute** `map (fun x => x * 2) [10; 42; 93; 2; 20].`

Exemples

- ▶ **Compute** `map (fun x => x * 2) [10; 42; 93; 2; 20].`
`= [20; 84; 186; 4; 40] : list nat`
- ▶ **Compute** `filter (fun x => x <=? 30) [10; 42; 93; 2; 20].`

Examples

- ▶ **Compute** `map (fun x => x * 2) [10; 42; 93; 2; 20].`
`= [20; 84; 186; 4; 40] : list nat`
- ▶ **Compute** `filter (fun x => x <=? 30) [10; 42; 93; 2; 20].`
`= [10; 2; 20] : list nat`
- ▶ **Compute** `map (fun x => x * 2)`
`(filter (fun x => x <=? 30) [10; 42; 93; 2; 20]).`

Examples

- ▶ **Compute** `map (fun x => x * 2) [10; 42; 93; 2; 20].`
`= [20; 84; 186; 4; 40] : list nat`
- ▶ **Compute** `filter (fun x => x <=? 30) [10; 42; 93; 2; 20].`
`= [10; 2; 20] : list nat`
- ▶ **Compute** `map (fun x => x * 2)`
`(filter (fun x => x <=? 30) [10; 42; 93; 2; 20]).`
`= [20; 4; 40] : list nat`
- ▶ **Compute** `map (fun x => x + 3)`
`(map (fun x => x * 10)`
`((filter (fun x => x <=? 30) [10; 42; 93; 2; 20]))).`

Examples

- ▶ **Compute** `map (fun x => x * 2) [10; 42; 93; 2; 20].`
`= [20; 84; 186; 4; 40] : list nat`
- ▶ **Compute** `filter (fun x => x <=? 30) [10; 42; 93; 2; 20].`
`= [10; 2; 20] : list nat`
- ▶ **Compute** `map (fun x => x * 2)`
`(filter (fun x => x <=? 30) [10; 42; 93; 2; 20]).`
`= [20; 4; 40] : list nat`
- ▶ **Compute** `map (fun x => x + 3)`
`(map (fun x => x * 10)`
`((filter (fun x => x <=? 30) [10; 42; 93; 2; 20]))).`
`= [103; 23; 203] : list nat`

Exemples (2)

- ▶ Fonction qui décide (dans `bool`) si un entier est pair :

Exemples (2)

- Fonction qui décide (dans `bool`) si un entier est pair :

```
Fixpoint evenb (n : nat) : bool :=  
  match n with  
  | 0 => true  
  | 1 => false  
  | S (S n) => even n  
  end.
```

- `Compute filter evenb [10; 42; 93; 2; 20].`

Exemples (2)

- Fonction qui décide (dans `bool`) si un entier est pair :

```
Fixpoint evenb (n : nat) : bool :=  
  match n with  
  | 0 => true  
  | 1 => false  
  | S (S n) => even n  
  end.
```

- `Compute filter evenb [10; 42; 93; 2; 20].`

```
= [10; 42; 2; 20] : list nat
```

- `Compute map (fun x => x * 10)
 (filter (fun x => x <=? 30)
 (filter evenb [10; 42; 93; 2; 20]))).`

Exemples (2)

- Fonction qui décide (dans `bool`) si un entier est pair :

```
Fixpoint evenb (n : nat) : bool :=  
  match n with  
  | 0 => true  
  | 1 => false  
  | S (S n) => even n  
  end.
```

- `Compute filter evenb [10; 42; 93; 2; 20].`

```
= [10; 42; 2; 20] : list nat
```

- `Compute map (fun x => x * 10)
 (filter (fun x => x <=? 30)
 (filter evenb [10; 42; 93; 2; 20]))).`

```
= [100; 20; 200] : list nat
```

map map

► **Lemma** `map_map` {A B C : Type} (f : B -> C) (g : A -> B)
 (l : list A) :
 map f (map g l) =

map map

- ▶ **Lemma** `map_map {A B C : Type} (f : B -> C) (g : A -> B)`
 `(l : list A) :`
 `map f (map g l) =`
 `map (fun x => f (g x)) l.`
- ▶ **Proof.**
 `induction l as [| h t IHt].`
 - `reflexivity.`
 - `simpl. rewrite IHt. reflexivity.`
 `Qed.`

filter filter

► **Lemma** filter_filter {A : Type} (p q : A -> bool) (l : list A) :
filter p (filter q l) =

filter filter

► **Lemma** filter_filter {A : Type} (p q : A -> bool) (l : list A) :
 filter p (filter q l) =
 filter (fun x => p x && q x) l.

filter filter

- ▶ `Lemma filter_filter {A : Type} (p q : A -> bool) (l : list A) :
 filter p (filter q l) =
 filter (fun x => p x && q x) l.`
 - ▶ `Proof.`
 - `induction l as [| h t IHt].`
 - `- reflexivity.`
 - `- simpl; destruct (q h); simpl;`
 `destruct (p h); rewrite IHt; reflexivity.`
- `Qed.`

Fonctions d'ordre supérieur : rappels

Rappels sur la composition

`fold`

Mieux composer

En OCaml :

- ▶ `let x = "foobar" |> String.length`
`(* val x : int = 6 *)`
- ▶ `let l = "foobar"`
 `|> (fun s -> s ^ "baz")`
 `|> String.split_on_char 'b';;`
`(* val l : string list = ["foo"; "ar"; "az"] *)`
- ▶ `let l = "foobar"`
 `|> (fun s -> s ^ "baz")`
 `|> String.split_on_char 'b'`
 `|> map (fun x -> String.length x);;`
`(* val l : string list = [3; 2; 2] *)`

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let ( |> ) =`

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (|>) =
 fun x -> fun f -> f x`
- ▶ Type?

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (|>) =
 fun x -> fun f -> f x`
- ▶ Type?
`val ( |> ) : 'a -> ('a -> 'b) -> 'b`
- ▶ Associativité à

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (|>) =
 fun x -> fun f -> f x`
- ▶ Type?
`val ( |> ) : 'a -> ('a -> 'b) -> 'b`
- ▶ Associativité à gauche.
- ▶ Voir : <https://ocaml.org/docs/operators>
- ▶ peu prioritaire (c'est ce qu'on veut) :
https://ocaml.org/manual/5.4/api/0caml_operators.html

Dans l'autre sens

- ▶ `let x = String.length @@ "foobar";;`
`(* val x : int = 6 *)`
- ▶ `let l = String.split_on_char 'b' @@ (fun s -> s ^ "bar")`
`@@ "foobar";;`
`(* val l : string list = ["foo"; "ar"; "ar"] *)`
- ▶ `let l = map String.length @@ String.split_on_char 'b'`
`@@ (fun s -> s ^ "baz") @@ "foobar";;`
`(* val l : int list = [3; 2; 2] *)`

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let ( @@ ) =`

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (@@) =
 fun f -> fun x -> f x`
- ▶ Type?

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (@@) =
 fun f -> fun x -> f x`
- ▶ Type?
`val ( @@ ) : ('a -> 'b) -> 'a -> 'b`
- ▶ Associativité à

Définition

- ▶ En vrai c'est primitif pour des raisons de performance (économiser un appel de fonction), mais c'est comme si :
- ▶ `let (@@) =
 fun f -> fun x -> f x`
- ▶ Type?
`val ( @@ ) : ('a -> 'b) -> 'a -> 'b`
- ▶ Associativité à droite.
- ▶ Voir : <https://ocaml.org/docs/operators>
- ▶ peu prioritaire (c'est ce qu'on veut) :
https://ocaml.org/manual/5.4/api/0caml_operators.html

En Rocq

Ces opérateurs ne sont pas définis. Le mieux est de définir nous-mêmes des notations :

Notation `"x |> f"` := (f x)

(at level 91, **left** associativity, only parsing).

Notation `"f @@ x"` := (f x)

(at level 90, **right** associativity, only parsing).

Fonctions d'ordre supérieur : rappels

Rappels sur la composition

`fold`

Exemple : somme des éléments d'une liste

- **Fixpoint** `sum_list l :=`
 `match l with`
 | `[] => 0`
 | `h :: t => h + (sum_list t)`
 `end.`
- `sum_list [1; 2; 3; 4] = 1 + (2 + (3 + (4 + 0)))`
- 0 est clairement le choix le plus naturel pour `sum_list []`
- Mais imaginons qu'on veuille une autre « valeur par défaut »

```
Fixpoint sum_list_with l start :=  
  match l with  
  | [] => start  
  | h :: t => h + (sum_list_with t start)  
  end.
```

- On prouve facilement :

```
Lemma sum_list_sum_list_with_0 l :  
  sum_list_with l 0 = sum_list l.
```

Proof.

```
  induction l as [| t h IH].  
  - reflexivity.  
  - simpl; rewrite IH; reflexivity.
```

Qed.

Somme des éléments d'une liste : autre possibilité

- On a défini

```
Fixpoint sum_list_with l start :=  
  match l with  
  | [] => start  
  | h :: t => h + (sum_list_with t start)  
  end.
```

- Mais on peut aussi se servir de `start` comme d'un accumulateur :

```
Fixpoint sum_list_acc l acc :=  
  match l with  
  | [] => acc  
  | h :: t => sum_list_acc t (acc + h)  
  end.
```

- `sum_list_acc [1; 2; 3; 4] 0 = (((0 + 1) + 2) + 3) + 4`

Produit des éléments d'une liste

```
Fixpoint prod_list_with l start :=  
  match l with  
  | [] => start  
  | h :: t => h * (prod_list_with t start)  
  end.  
Compute prod_list_with [1; 2; 3; 4] 1.  
Fixpoint prod_list_acc l acc :=  
  match l with  
  | [] => acc  
  | h :: t => prod_list_acc t (acc * h)  
  end.
```

Généralisations (OCaml)

En OCaml :

```
let rec fold_right f l accu =  
  match l with  
  | [] -> accu  
  | a::l -> f a (fold_right f l accu)
```

de type

```
val fold_right : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```

On se rend compte que finalement les arguments de l'opération binaire n'ont en fait pas besoin d'être sur le même type. Correspond intuitivement à

$$f\ l_0\ (f\ (l_1\ (f\ l_2\ \dots(f\ l_{n-1}\ accu)))\dots)$$

```
let rec fold_left f accu l =  
  match l with  
  | [] -> accu  
  | a::l -> fold_left f (f accu a) l
```

de type

```
val fold_left : ('a -> 'b -> 'a) -> 'a -> 'b list -> 'a
```

Correspond intuitivement à

$$f\ (f\ (f\ (\dots(f\ accu\ l_0)\ l_1)\ l_2\ \dots)\ l_{n-1})$$

Généralisations (Rocq)

```
Fixpoint fold_right {A B : Type} (f : B -> A -> A) (accu : A)
  (l : list B) : A :=
  match l with
  | [] => accu
  | b :: t => f b (fold_right f accu t)
  end.

Fixpoint fold_left {A B : Type} (f : A -> B -> A) (accu : A)
  (l : list B) :=
  match l with
  | [] => accu
  | b :: t => fold_left f (f accu b) t
  end.
```